

LECTURE 1

Introduction to Databases and Information Systems

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

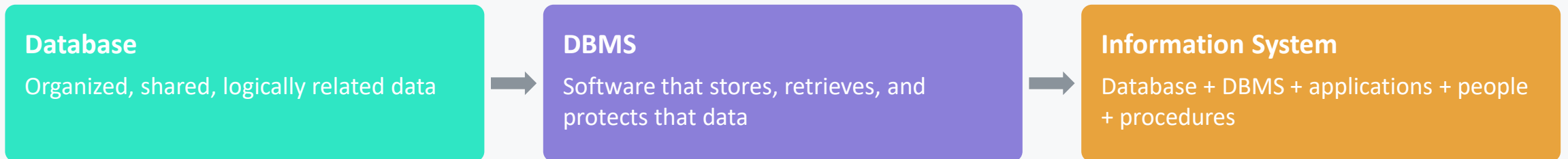
- What a database is, and how it differs from an information system built around one
- Common categories of database applications you already use
- How data management evolved from manual filing to file-based systems to databases
- File-based systems: how they worked, and the problems that forced a rethink
- The database approach: what changes, concretely, when data is centralized
- The advantages a DBMS gives you — and the real costs it imposes in return

Databases and Information Systems

A **database** is a shared, organized collection of logically related data, designed to meet an organization's information needs.

- Shared — many users and programs draw on the same data, not private copies
- Organized — structured so it can be found, related, and reasoned about
- Logically related — records about the same real-world entity are known to be connected

From data to an information system



Database vs. Spreadsheet

- A spreadsheet stores data too — but it has none of a DBMS's built-in guarantees
- No enforced structure: nothing stops a “Student ID” column from holding text, duplicates, or values with no matching student
- No safe concurrent access: a DBMS lets 50 people update the same data at once without corrupting it; a shared spreadsheet file cannot
- No crash recovery: a DBMS restores a consistent state after a crash; a spreadsheet application offers no such guarantee
- Rule of thumb: if multiple people must trust the same numbers at the same time, it belongs in a database, not a spreadsheet

Database Applications

Domain	What's stored	Example
Banking	Accounts, balances, transactions	Approving a debit card transaction
Airlines	Reservations, schedules, seat maps	Confirming a seat on flight PK-301
University records	Students, courses, enrollments, grades	Producing a graduate's transcript
Telecom	Call records, subscriber accounts	Generating a monthly phone bill
Sales / e-commerce	Products, orders, customers, inventory	"Only 3 left in stock"
Human resources	Employees, salaries, positions	Running monthly payroll

A quick test: is this really a database problem? If the data must be shared by more than one person or program, stay accurate under concurrent access, and survive a crash without loss — it belongs in a database, not a private file.

Is This Really a Database Problem?

- The three-part test: shared by more than one person or program; must stay accurate under concurrent updates; must survive a crash without silent loss
- Case A — a professor's personal sign-up sheet that no one else reads: fails all three conditions, so a plain file is fine
- Case B — the same sign-up sheet, now also read by the department office and by TAs to confirm attendance: now shared and must stay consistent, so it belongs in a database
- The test is about consequences, not size: a small table many people rely on at once needs a database; a huge file only one program ever touches may not
- Banking, airlines, and university records all pass this test immediately — that is exactly why they are database applications

Evolution of Data Management

Data management evolved in stages, each responding to the weaknesses of the one before it.

Manual / paper filing

Ledgers, index cards, filing cabinets — slow, error-prone, one copy



File-based systems

Each application owns its own files; no coordination between them



Database approach (DBMS)

One managed, shared collection of data serving every application

Why Focus on File-Based Systems First?

- Memorizing the three stages — manual filing, file-based systems, the database approach — is not the point of this lecture
- What matters is understanding why each stage failed, because that failure is exactly what the next stage was built to fix
- File-based systems reward close study: structured enough to resemble a database, yet broken in ways a database directly solves
- The next few slides stay inside the file-based world and look hard at exactly how it breaks, before showing how the database approach fixes it

File-Based Systems: A University Before Databases

Suppose COMSATS ran three separate offices, each with its own file-based system, all needing student data:

Registrar's file

RegNo, Name, CNIC, Address, Program

Library's file

RegNo, Name, Address, BooksIssued

Finance office's file

RegNo, Name, Address, FeeBalance

Name and Address are stored three separate times — once per office, each written by its own program. That single decision is the root of almost every problem file-based systems became known for.

- Data redundancy — the same facts duplicated across every file that needs them
- Data inconsistency — independent copies drift apart with no way to tell which is current
- Poor data sharing — files were never designed to talk to each other
- Program-data dependence — physical file layout is hard-coded into each program

Worked Example: Tracing an Inconsistency

When	What Happens	State of the Three Copies
Day 1	All three offices' files are created together	All three show the same address
Week 6	Student moves; tells only the Registrar	Registrar updated; Library & Finance stale
Week 10	Library mails a notice to the file address	Sent to the OLD address — never arrives
Week 14	Finance computes a late fee from its copy	Fee is wrong — the move was unrecorded
Week 16	Student applies for a transcript	Registrar and Finance now disagree
Result	Three independent “truths,” one real fact	No file alone knows which copy is current

Why this matters: Three files, one fact, and no single place to look for the truth — this is exactly the inconsistency problem file-based systems cannot solve on their own.

Poor Sharing & Program-Data Dependence

- Poor data sharing — the library's program cannot simply ask the registrar's file “is this student still enrolled?”; the files were never designed to talk to each other
- Program-data dependence — a file's physical layout (field order, field sizes, data types) is hard-coded directly into the program that reads it
- Worked consequence: the registrar decides to add a Major field to its student file
- Every program that reads that file — not just the registrar's own software — must be found, rewritten, and recompiled before anything works again
- In a database, the same change is one instruction to the DBMS; programs that don't care about Major are not affected at all

File-Based Systems: Why It Broke Down

- Limited sharing, unanticipated queries — “students who owe a library fine AND a tuition balance” spans two incompatible files; answering it means writing a brand-new program
- Integrity and security are hard to enforce consistently — each program enforces its own rules (or doesn't); there's no single place to say “a fee balance can never be negative”

Inconsistency Is the Most Dangerous Failure

- Step 1 — a student pays their entire outstanding tuition balance at the finance office in week 10
- Step 2 — the finance office updates only its own file; no other office's file is touched, because the files don't talk to each other
- Step 3 — in week 14, the student applies for their transcript at the registrar's office
- Step 4 — the registrar's file still shows the balance from before the payment; it was never told the payment happened
- Step 5 — the transcript request is blocked, even though the debt no longer exists, because the stale copy looks just as “official” as the current one
- Why this is worse than redundancy: wasted storage is a visible, budgetable cost; a wrong “official” answer is invisible until it blocks someone

The Database Approach

Instead of every application owning a private copy of the data it needs, **store the data once, centrally, and let every application go through a DBMS to reach it.**

File-based (before)

Registrar, Library, and Finance each keep their own Name/Address copy — three files, three truths

Database approach (after)

One Student table holds Name/Address once; every office reads the same row through the DBMS

Worked example: a single Student relation

RegNo	Name	Address	Program
FA21-BCS-001	Ali Raza	Model Town, Lahore	BS Computer Science

The library's and finance's systems no longer keep their own Name/Address copies — they store only BooksIssued / FeeBalance and reference the same shared row through the DBMS.

Worked Example: Rebuilding the Student Table

Table	Columns It Stores	What Changed
Student (one copy)	RegNo, Name, Address, Program	Lives in one place, via the DBMS
Library's own data	RegNo (reference), BooksIssued	No Name/Address — lives only in Student
Finance's own data	RegNo (reference), FeeBalance	No Name/Address — same shared reference
Updating an address	Change Student.Address once	Library and Finance see it immediately
Cross-office query	Fine AND balance, one query	Joins on RegNo — no new program needed
Compare to before	Same question, file-based world	Needed a brand-new program from scratch

Note what disappeared: no duplicate Name or Address columns anywhere outside the Student table — redundancy is gone by construction, not by discipline.

Advantages of the Database Approach (1/2)

- **Control of data redundancy** — store each fact once wherever possible
- **Data consistency** — with redundancy controlled, copies can't disagree
- **More information from the same data** — combining files answers questions no single file could
- **Sharing of data** — existing data can be reused by new applications
- **Improved data integrity** — validation rules are defined once and enforced everywhere
- **Improved security** — access restricted per-user, down to individual columns

Advantages of the Database Approach (2/2)

- **Enforcement of standards** — date and ID formats enforced centrally, not per-programmer
- **Economy of scale** — one managed system costs less overall than many small ones
- **Balance of conflicting requirements** — a DBA arranges data for the organization as a whole
- **Improved accessibility and responsiveness** — query languages answer unanticipated questions directly
- **Increased productivity** — the DBMS provides storage, retrieval, concurrency, and recovery
- **Improved backup and recovery** — far more robust than one programmer's ad hoc script

Advantages in Action: Applying Them to COMSATS

- Improved security, concretely — a library clerk's login can be restricted to BooksIssued, with no access to another student's FeeBalance
- Improved integrity, concretely — one rule, “FeeBalance can never be negative,” is defined once and applies to every application automatically
- More information from the same data — one query across the shared Student table answers “who owes a library fine and a tuition balance?” directly
- Balance of conflicting requirements — a DBA tunes the shared database for the registrar, library, and finance office together, not for one office alone
- Every one of these was impossible, or required a brand-new custom program, back when the three offices kept separate files

Disadvantages of the Database Approach

The database approach is not free — it trades one set of problems for another, smaller but real:

Complexity

Designers, developers, DBAs, and end users all need to understand enough of the DBMS to use it well

Cost

DBMS software and the hardware it demands add significant up-front and ongoing expense

Reduced performance for narrow needs

A general-purpose DBMS can be slower than a program built for exactly one task

Higher impact of a failure

A centralized, shared database means an outage affects every dependent application at once

Why Organizations Adopt the Database Approach Anyway

- None of the four disadvantages — complexity, cost, narrower performance, bigger blast radius — are usually enough on their own to justify going back to file-based systems
- The redundancy and inconsistency problems of file-based systems only get worse as an organization grows; they never self-correct
- The database approach's costs are largely one-time: learning the technology, paying for the software and hardware up front
- Exception: a tiny, single-user, single-purpose tool with no sharing requirement may genuinely be better served by a plain file
- It is a trade-off, not a universal law — but for any shared, growing dataset, the trade-off almost always favors the database

From File Chaos to a Managed Database

- We started with three COMSATS offices, each keeping its own private file, each with its own copy of Name and Address
- Those duplicated copies drifted apart — the exact mechanism behind data redundancy and data inconsistency
- The database approach's fix: store each fact once, centrally, and route every application through a DBMS to reach it
- That one change unlocked consistency, integrity, security, and shared access — at the cost of added complexity and a bigger blast radius on failure
- Next: Lecture 2 opens up what a DBMS is actually made of, and who is responsible for a database once an organization adopts this approach

Key Takeaways

- A database is shared, organized, logically related data; a DBMS manages it; an information system is the database, DBMS, and applications together
- Databases quietly power banking, airlines, university records, telecom, sales, and HR systems
- File-based systems gave each application its own private files, causing redundancy, inconsistency, poor sharing, and program-data dependence
- The database approach replaces many private files with one centrally managed, shared pool of data accessed through a DBMS
- The database approach earns real advantages — consistency, integrity, security, standards, and recovery — at the cost of complexity, expense, and a bigger blast radius on failure

Looking Ahead: Lecture 2

- Lecture 2 picks up exactly where this one stops: the database approach's environment, roles, and components
- You'll meet the people responsible for a real database — the DBA, application programmers, and end users — and what each one does
- You'll see the DBMS's own internal components: the query processor, the storage manager, and the catalog that describes the data itself
- Everything in today's lecture — why centralize data, what it buys you — becomes the motivation for everything Lecture 2 names and defines